A Comparative Evaluation of Agentic AI Systems for Data Analysis: LAMBDA, ChatGPT, and Copilot

1. Introduction

Large language models have made natural language–driven data analysis increasingly accessible. Tools like ChatGPT allow users to perform exploratory analysis, build predictive models, and interpret results without writing code. While these systems offer convenience and flexibility, they often operate as opaque, single-agent assistants. Their analytical decisions are not always traceable, and their outputs are rarely validated by design—posing challenges for reproducibility, transparency, and auditability.

Agentic AI systems represent a different approach. By assigning modular roles to specialized agents—such as planning, coding, and validation—they aim to structure multi-step workflows in a more reliable and interpretable manner. LAMBDA is one such system, designed to respond to high-level analytical prompts through coordinated agent behavior and tightly coupled execution.

This study investigates how LAMBDA performs in practice, compared to two widely used tools: ChatGPT and GitHub Copilot. Rather than evaluating correctness alone, we examine how architectural design influences system behavior across repeated tasks. We focus on differences in consistency, adaptability, and responsiveness to user intent—particularly under incomplete or ambiguous instructions. Through this comparison, we ask whether agentic structure meaningfully improves trust and reliability in automated analysis.

2. Agentic AI and Motivating Principles

Large language models such as ChatGPT have become popular tools for data analysis, yet their single-agent, generative nature offers limited support for multi-step task decomposition, intermediate validation, and stateful orchestration. These limitations have motivated the rise of agentic AI systems, which distribute reasoning and execution across modular roles.

Recent work surveying large language model–based autonomous agents identifies planning, memory, tool use, and iterative feedback loops as key mechanisms for improving reliability and traceability beyond monolithic LLM usage (Wang et al., 2023). Complementary research on multi-agent LLM systems emphasizes coordination among specialized agents as a way to support complex, multi-step tasks through structured interaction rather than free-form prompting (Tran et al., 2025). Building on these insights, this study empirically characterizes the behavior of LAMBDA, a multi-agent data analysis system, in contrast with ChatGPT and GitHub Copilot—two general-purpose LLM tools that operate without persistent agent structure, explicit role decomposition, or orchestration logic.

Rather than evaluating output correctness alone, our focus is on how system architecture influences analytical behavior: including consistency, flexibility, execution logic, and responsiveness to user intent. We analyze each system across repeated runs on a shared task list, using a rubric to assess key behavioral dimensions. These behavioral dimensions—consistency, flexibility, and responsiveness—serve as the basis for both our experimental design (Section 4) and our structural interpretation of LAMBDA's behavior (Section 5).

3. LAMBDA Overview

LAMBDA (Sun et al., 2024) is a multi-agent data analysis system that enables users to perform statistical workflows through natural language prompts. The interface allows users to upload datasets, request analyses, and receive generated code, intermediate outputs, and execution feedback—all without writing or running code manually.

The system's agentic architecture separates task orchestration, code generation, and validation into modular roles. This enables LAMBDA to handle multi-step workflows more robustly, with automatic execution and error exposure built into its loop. Users interact via a browser-based chat interface, issuing instructions incrementally or as full task lists.

While the full system design is documented in Sun et al. (2024), this report examines how LAMBDA's architecture influences its analytical behavior in practice, especially in comparison to general-purpose tools like ChatGPT and GitHub Copilot. The next sections investigate how these systems differ in consistency, adaptability, and responsiveness when given the same analytical tasks.

4. Experimental Design and Observed Behavior Comparison

4.1 Objective

This section evaluates how LAMBDA, ChatGPT, and GitHub Copilot behave under the same end-to-end analytical pipeline. The aim is not predictive performance but system behavior: how each tool interprets vague instructions, handles ambiguity, constructs workflows, and manages state across steps. We focus especially on whether modeling choices, data splits, and parameter settings persist across steps without explicit user intervention.

Our evaluation targets the following questions:

- How much initiative does each system take in building a defensible workflow?
- Can it adapt to user instructions without compromising internal consistency?
- Do its responses remain auditable, interpretable, and reusable?

The comparison centers LAMBDA's agentic execution structure and contrasts it with ChatGPT's single-agent conversational flow and GitHub Copilot's code-completion approach.

4.2 Experimental Setup and Prompt Sequence

All experiments were conducted on the same dataset: the Stats + Stories 300th Episode Data Visualization Contest dataset, which contains a mix of numerical, categorical, and date variables related to podcast episodes. Its moderate complexity makes it suitable for evaluating realistic preprocessing, modeling, and interpretation workflows.

All three systems were powered by OpenAI language models: LAMBDA internally uses gpt-5-mini within its agentic execution framework, while ChatGPT and GitHub Copilot were accessed using gpt-5.2.

We designed a twelve-prompt sequence to simulate a realistic data analysis pipeline, intended to test systems' continuity, autonomy, and reuse of prior steps:

1. Load the dataset and perform any necessary preprocessing.
2. Perform exploratory data analysis to understand key patterns in the data.
3. Split the data into training and test sets with a proportion of 8:2.
4. Select relevant features for modeling.
5. Train a logistic regression model and evaluate it by accuracy on the test set.
6. Train an SVM model and evaluate it by accuracy on the test set.
7. Train an MLP model and evaluate it by accuracy on the test set.
8. Train a decision tree model and evaluate it by accuracy on the test set.
9. Train a random forest model and evaluate it by accuracy on the test set.
10. Use GridSearchCV to find the best hyperparameters for the MLP model (five parameter groups).
11. Save all trained models.
12. Generate a report summarizing the analysis.

Prompts were issued sequentially, without explicit instructions to reuse intermediate variables, objects, or results unless the system chose to do so autonomously. Although phrased independently, the prompt sequence implies a clear dependency structure. In particular, downstream modeling steps (Steps 5–9) are expected to rely on the train/test split established in Step 3 and the feature set selected in Step 4, and hyperparameter tuning (Step 10) is expected to operate on the same model and data representation defined earlier.

This design allows us to evaluate whether each system preserves and respects analytical dependencies across steps, or instead treats each prompt as an isolated execution unit. Observed behavior in later sections is interpreted relative to this expected dependency structure.

To evaluate reproducibility and variability, we ran **five separate 8:2 conversational runs** for both LAMBDA and ChatGPT. In addition, a **sixth run using a 70:30 split** was included to test whether split ratios propagate through later steps or revert to default configurations. All runs were manually reviewed and logged.

Each system's behavior was assessed not just for correctness, but for the logic and consistency of its choices:

- How data splits and preprocessing were performed and reused
- Whether modeling steps carried forward prior variables or reset them
- How faithfully hyperparameters were tuned
- Whether models were preserved or recomputed later
- How much explanation was given for each decision

GitHub Copilot was also tested on the same twelve tasks using a single all-at-once input cell. However, due to its lack of interactivity and task segmentation, it failed to complete all tasks in step-wise conversational format and is only included in our all-at-once task evaluation.

At the end of this section, we evaluate each system against a ten-point rubric, introduced in Section 4.4.

4.3 Behavioral Observations Across Prompts

To examine how each system handled end-to-end analytical tasks, we tracked the autonomous decisions made at each of the twelve prompt steps. We evaluated how the dataset was loaded and preprocessed, which exploratory visualizations were produced, what train-test split strategies were used (including whether earlier splits were reused), how features were selected, whether models honored prior configurations, how tuning and saving were handled, and how summaries were presented.

A full comparative breakdown is provided in **Table 1**, which summarizes the decisions made by LAMBDA and ChatGPT across five repeated runs per tool. LAMBDA's outputs were consistent across runs and are shown jointly; ChatGPT's results reflect observed variability. GitHub Copilot is excluded from this table due to its failure to support step-wise prompting and is instead evaluated in **Section 4.5**.

Table 1. Analytical decisions made by LAMBDA and ChatGPT under identical prompt sequences.

Prompt Step	LAMBDA Summary	ChatGPT Summary
1. Load data & preprocessing	Consistently drops missing values and coerces types. Pipeline reuse varies slightly across runs.	Also drops missing values, but more variation in whether types are coerced and whether one-hot encoding is applied. Column naming is more consistent.
2. EDA	Bar plots most common, but some runs include box plots or line charts. Less auto-commentary. Plots may differ by run.	Produces correlation heatmaps, bar charts, and boxplots. Explains trends (e.g., variable relationships) more often. Visualization types are consistent across runs.

Prompt Step	LAMBDA Summary	ChatGPT Summary
3. Train-test split	Split is sometimes redone later. Variable names (`train_data`, `X_train`, etc.) can vary across steps. Stratification inconsistently mentioned.	Usually reuses split objects (`X_train`, `X_test`). Mentions random_state and stratify more often. Less variation in naming.
4. Feature selection	Sometimes uses all features, sometimes selects subset. Text features rarely used. Can ignore user input in minor ways (e.g., type of encoding).	Tends to select numeric columns explicitly. Strategy can vary by run (sometimes manual selection, sometimes automatic).
5. Logistic regression	Uses `liblinear` or default solver. Occasionally reuses `X_train`, sometimes retrains from full dataset.	More consistent in using `lbfgs` (default). Often forgets to reuse preprocessing pipeline. Variable reuse more stable.
6. SVM	Kernel default (rbf) used. Retrains model without reusing prior train/test split in some runs. Sometimes introduces new preprocessing steps.	Uses rbf or linear kernel. Reuses split more often. Hyperparameters like `C` left at default.
7. MLP	Tends to use `max_iter=300` or `500`, sometimes varies per run. Convergence warnings appear. Good integration with GridSearch in later prompt.	Similar parameter ranges, but inconsistency in activation function, solver, and convergence. Sometimes retrains from scratch without reuse.
8. Decision tree	Depth constraints rarely specified. Defaults often used. Reports feature importance if prompted.	Varies across runs: sometimes applies `max_depth`, sometimes uses defaults. Reports accuracy but rarely explains decision rules.

Prompt Step	LAMBDA Summary	ChatGPT Summary
9. Random forest	Typically uses `n_estimators=100`. Random_state applied. Model saved correctly if instructed.	More variation in number of estimators. Sometimes forgets to specify seed. Tends to reuse train/test split better.
10. Hyperparameter tuning (MLP)	Executes GridSearchCV effectively with defined parameter grid. Results saved and reused in some runs.	Can execute GridSearch, but parameter grid inconsistent across runs. Often reinitializes model. Less reuse of best estimator.
11. Save models	Only saves models when explicitly instructed. Earlier models not retained after cache reload. Auto-download option appears per model.	Does not persist models across sessions. Saved models must be manually prompted for. Cannot generate download links.
12. Report generation	Generates full markdown reports with summary stats, accuracy tables, and pipeline descriptions. Rich content if all steps are completed in one run.	Report generation must be manually prompted. Output is more like a code and result summary, not a formatted multi-section report.

4.4 Rubric-Based Evaluation of Observed Behavior

To systematically assess the systems' capabilities beyond individual behaviors, we applied a ten-criterion rubric grounded in reproducibility, flexibility, auditability, and human interpretability. The rubric evaluates the following:

1. **Ease of getting started:** How quickly and intuitively the tool begins the analysis without requiring complex setup or lengthy instructions.
2. **Ease of iterating model choices:** How easily the underlying model type can be changed without restarting or restructuring the workflow.
3. **Ability to complete the task:** Whether the tool completes the full analytical pipeline correctly and defensibly, from data loading through model training and reporting.
4. **Statefulness:** Whether the system keeps the data, models, and other results it created earlier so they can still be used or saved in later steps of the same session.

5. **Ease of tuning model parameters:** How easily hyperparameters or modeling assumptions can be modified without restarting the workflow.
6. **Stability (Data Decisions):** Consistency of preprocessing decisions (e.g., missing values, encoding, feature inclusion) across repeated runs.
7. **Stability (Method Choices):** Consistency of modeling decisions (e.g., scaling, solver choice, model defaults) across repeated runs under identical prompts.
8. **Continuity across steps:** The ability to build later steps directly on earlier ones within a single workflow, without redoing or contradicting previous work.
9. **Transparency / Auditability:** The degree to which the system shows what it did, why it did it, and what intermediate states looked like.
10. **Interpretability of outputs:** Clarity of explanations, code structure, and reporting for a human reader.

Each system received a score from 1 to 5 per criterion. Ratings are based on direct evidence from all six runs, including analysis of intermediate code, variable naming, reuse of earlier steps, and decision-making transparency. These scores are summarized visually in **Figure 1** and numerically in **Table 2**.

Note: GitHub Copilot is excluded from this rubric evaluation, as it could not support sustained conversational state across sequential prompts. Its performance is analyzed separately under task-list execution (Section 4.5).

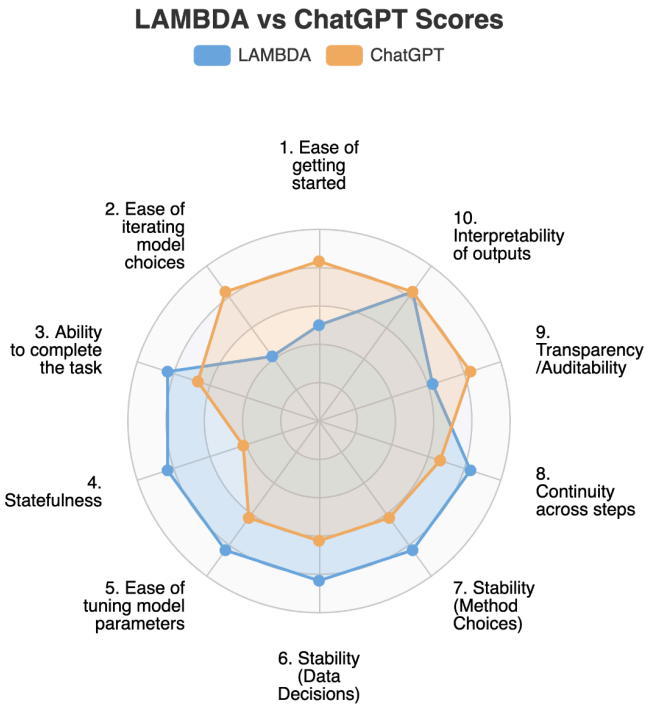

Figure 1. Radar chart of LAMBDA vs ChatGPT scores across 10 evaluation criteria.

Table 2. Rubric-based evaluation scores for LAMBDA and ChatGPT across ten criteria.

Criterion	LAMBDA	ChatGPT	Justification Summary
1. Ease of getting started	**3**	**5**	LAMBDA requires the user to clone the repo, install Python dependencies, and run the app in a local or Codespaces environment, which involves some setup overhead. ChatGPT launches instantly from a browser with no installation or version concerns.
2. Ease of iterating model choices	**2**	**4**	LAMBDA requires full re-prompting or an all-at-once task list to switch models; it does not persist modeling objects across steps unless explicitly redefined. It often renames splits, resets assumptions, or reruns preprocessing. ChatGPT allows model changes with minimal friction mid-session (e.g., “now use random forest”) and tends to follow those instructions directly.
3. Ability to complete the task	**5**	**4**	LAMBDA completes the entire modeling pipeline (from data load to evaluation), including model saving and optional tuning, without error across runs. ChatGPT also finishes the workflow, but sometimes it may apply a new model to unscaled data if preprocessing wasn't preserved across steps.
4. Statefulness	**4**	**2**	LAMBDA tracks and saves models and data across prompts during a live session, and allows saving specific models on request. However, resuming from cache resets model memory. ChatGPT can recall previous steps conversationally, but cannot preserve or reload trained models or files.
5. Ease of tuning model parameters	**4**	**3**	LAMBDA handles GridSearchCV-style tuning well and integrates it into its structured pipeline. It applies changes reliably when told. ChatGPT accepts parameter tuning (e.g., changing solver or layers) but often forgets prior settings or applies them inconsistently unless reminded.

Criterion	LAMBDA	ChatGPT	Justification Summary
6. Stability (Data Decisions)	**4**	**3**	LAMBDA makes consistent decisions on missing values, encoding schemes, and feature selection across repeated runs. ChatGPT's preprocessing behavior can vary subtly depending on how the prompt is phrased (e.g., whether it imputes or drops nulls).
7. Stability (Method Choices)	**4**	**3**	LAMBDA generally uses the same solvers, activations, and model defaults unless told otherwise. ChatGPT sometimes changes model behavior (e.g., solver, layer sizes) across runs or fails to state what changed.
8. Continuity across steps	**5**	**4**	LAMBDA retains data splits, model objects, and preprocessing pipelines consistently throughout the session, as long as it's not resumed from cache. ChatGPT maintains variable context within a session but can silently reinitialize models or reprocess data unless reminded.
9. Transparency / Auditability	**3**	**4**	LAMBDA logs each step clearly and uses consistent naming, but rarely explains why decisions were made unless explicitly prompted. Most of its transparency comes from the final report rather than in-session chat. ChatGPT often explains its reasoning in-line without needing to be asked, making it easier to follow and audit the analysis as it unfolds.
10. Interpretability of outputs	**4**	**4**	LAMBDA's outputs are clean and labeled, but in-session results tend to be minimal and require the final report for full context. It presents metrics clearly but with little interpretive commentary. ChatGPT gives more narrative context in-session, helping users understand what the numbers mean, though its formatting can vary.

Detailed justifications and example excerpts from the run logs are available in **Appendix A**.

4.5 Task-List (All-at-Once) Execution

In addition to the step-by-step prompt sequence used in prior evaluations, we tested each tool's ability to autonomously execute a complete analysis pipeline from a single task list. This mode assesses how well the system interprets and manages an end-to-end workflow when the user provides all instructions upfront, without incremental guidance. It also reveals differences in agentic behavior when systems are tasked with reasoning through an entire pipeline independently.

LAMBDA, ChatGPT, and GitHub Copilot were each given the same 12-prompt sequence used in prior evaluations, but delivered as a single all-at-once task list rather than step-by-step.

LAMBDA

LAMBDA handled the full task list with minimal need for clarification. It correctly executed all 12 instructions in order, maintained coherent variable reuse across steps, and generated a final report consistent with its prior conversational outputs. The system dynamically selected appropriate hyperparameter grids for the MLP tuning step and completed the task without re-prompting. Notably, this autonomous performance was *stronger* than some of its step-wise runs, likely due to LAMBDA's internal orchestration framework being optimized for full-pipeline planning. Its ability to maintain state, structure execution logically, and generate a clean report suggests that the task-list mode aligns well with its design.

ChatGPT

ChatGPT also completed most of the task list accurately, handling data preprocessing, modeling, and evaluation without major breakdowns. It reused consistent variable names and largely preserved logic across steps. However, the quality of its explanations and report was poorer than LAMBDA's: it provided inline summaries rather than a structured document, and it occasionally skipped minor cleanup steps (e.g., dropping unused variables or saving models). Still, ChatGPT's flexibility in accepting a complex list without confusion highlights its robustness, even without a dedicated agent framework.

GitHub Copilot

In contrast, GitHub Copilot failed to execute the same task list autonomously. Despite embedding the 12-step prompt as markdown comments inside a Python notebook, Copilot progressed only partially through the pipeline. It generated runnable code up to Prompt 10 but stalled at Prompt 11 (MLP hyperparameter tuning), where it encountered an execution error. The agent failed to resolve the issue or proceed to later steps like model saving and report generation. Furthermore, the notebook became stuck in a "Retrieving Notebook summary…" state, and repeated instructions to rerun the full workflow were ignored. This failure mode mirrors what occurred under step-wise prompting: each new command triggers summary refresh attempts, and the agent repeatedly fails to recover or continue. These issues highlight that Copilot is fundamentally designed for reactive line-level code suggestions, not orchestrating or executing multi-step analytical pipelines in a notebook interface.

Interpretation

This task-list setting reveals important differences in autonomy and execution logic across systems. LAMBDA, with its agentic backbone and internal decision framework, excelled in planning and executing a complete workflow when given a high-level task. ChatGPT showed general-purpose competence but lacked structure in reporting and robustness in execution. Copilot, while powerful for localized coding assistance, lacked the initiative to carry out an end-to-end process.

Together with the step-wise experiments, these results show that LAMBDA improves when executing holistically, whereas ChatGPT remains more stable across modes, and Copilot is unusable outside of reactive coding contexts.

5. Why LAMBDA Behaves the Way It Does

LAMBDA's behavior across all five runs reflects core principles built into its agentic design. The system prioritizes reproducibility, successful execution, and modular responsibility across its agents. These architectural choices shape how it interprets user instructions, handles task decomposition, and navigates runtime decisions. This section explains the mechanisms behind LAMBDA's behavior and how they differ from those of general-purpose systems like ChatGPT and GitHub Copilot.

5.1 Execution Control and Validation Strategy

The core of LAMBDA is a validation-driven execution loop. Instructions are parsed by the Orchestrator, then passed to the Programmer for code generation. The Inspector evaluates whether the generated code runs without errors. If execution is successful, the system accepts the result and moves forward.

Importantly, LAMBDA includes limited retry functionality. If a code block fails to execute, the system may regenerate it up to a predefined number of attempts. However, these retries remain scoped to the original instruction—they do not represent a search for fundamentally different strategies. For example:

- If `GridSearchCV` fails due to parameter shape or input type, the Programmer may adjust formatting or argument order.
- But LAMBDA does not substitute a different model, nor does it pivot to a new analytical approach unless explicitly prompted.

Success is defined by clean execution, not by evaluating multiple approaches and selecting the best. This is in contrast to ChatGPT, which may revise its response if prompted further or if it detects an inconsistency in prior output. Copilot, on the other hand, lacks any execution layer altogether and only produces context-dependent code completions.

5.2 Prompt Granularity and Step Isolation

LAMBDA processes each prompt as a standalone instruction. While objects like variables may persist across steps if they are named consistently, the system does not retain higher-level analytical intent across prompts unless explicitly re-specified.

This prompt isolation leads to several observable behaviors:

- In step-wise mode, LAMBDA occasionally redefines preprocessing steps, train-test splits, or variable encodings, even if those steps were already completed earlier.
- In task-list mode, execution tends to proceed more smoothly, as dependencies are laid out in a single instruction and orchestrated more cleanly.
- Despite these differences in input format, the final pipelines remain structurally similar. The underlying agent behavior does not differ between modes.

ChatGPT, by contrast, relies on conversational history as implicit context. It may skip previously completed steps or infer that certain operations are redundant. While this enables more fluid back-and-forth, it also introduces variability across runs. Copilot operates on an even narrower context window and does not track prior completions unless visible in the code buffer.

LAMBDA's strict prompt isolation reinforces execution stability but limits dynamic coordination between analytical stages.

5.3 Hyperparameter and Model Configuration Behavior

LAMBDA's handling of hyperparameters and model training reflects its procedural, template-based architecture. Once a modeling instruction is received, the Programmer generates code based on predefined scaffolds, with fixed tuning structures and default parameters unless otherwise specified.

Across all five runs:

- `GridSearchCV` was invoked using consistent structure (e.g., 3-fold CV, accuracy scoring).
- The range of parameter values varied slightly, likely due to internal randomness, but stayed within narrow, pre-validated bounds.
- There was no evidence that LAMBDA adjusted search space size, selected new metrics, or adapted tuning based on performance results.

This behavior differs from ChatGPT, which occasionally omits tuning or varies the search scope based on prompt phrasing. Copilot never initiates hyperparameter tuning unless scaffolded manually, and cannot coordinate multi-step configuration.

In terms of model selection:

- LAMBDA does not propose alternative models unless they are included in the task prompt.

- If prompted to train logistic regression, SVM, MLP, decision tree, and random forest, it executes each in sequence using internal templates.
- Even when exploratory analysis is included earlier in the workflow, those insights are not used to modify the modeling sequence.

These decisions are not dictated by performance logic, but by instruction-following. LAMBDA does not compare models or revise its plan based on accuracy differences unless explicitly instructed to do so.

5.4 Architectural Trade-Offs and Implications

The consistency observed in LAMBDA's behavior stems directly from its system architecture. The division of roles across agents (Orchestrator, Programmer, Inspector) enables the system to construct pipelines that are stable and reproducible—but also rigid.

This results in the following trade-offs:

- **Strengths**:
    - High reproducibility across runs and prompt modes
    - Clear error recovery within single-task execution
    - Reliable adherence to instruction structure
- **Limitations**:
    - No data-aware adaptation of model or tuning logic
    - Limited responsiveness to intermediate analytical results
    - Redundant computation under step-wise prompting unless manually optimized

In contrast:

- ChatGPT enables more adaptive behavior, particularly in conversational settings, but often lacks reproducibility without explicit guardrails.
- Copilot supports rapid code completion but requires human orchestration to produce coherent workflows.

LAMBDA operates more like a domain-specific pipeline executor than a general-purpose assistant. It does not re-evaluate prior steps, synthesize across prompts, or choose between alternative analytical paths. These are not bugs or oversights—they are direct consequences of its architecture.

For users prioritizing auditability, instruction alignment, and reproducibility, LAMBDA offers a structured alternative to generative tools. But where exploratory iteration or data-informed model evolution is needed, its current design imposes meaningful constraints.

6. Future Work

This evaluation highlights several opportunities to extend LAMBDA's architecture and analytical capabilities.

- **Incorporating Analytical Feedback Loops**
  LAMBDA treats successful code execution as a sufficient stopping point. However, it does not evaluate the quality or relevance of the output unless explicitly instructed. Introducing a mechanism that allows agents to compare candidate models or strategies based on performance metrics would support more informed decision-making.
- **Cross-Prompt Context Awareness**
  LAMBDA treats each user prompt as an isolated instruction and does not retain awareness of what has been done previously unless the user restates it. As a result, the system may repeat steps or redefine variables unnecessarily. Future versions could improve the Orchestrator by adding lightweight memory or context-tracking capabilities, allowing it to recognize prior actions and maintain continuity across multi-step workflows.

- **Expanded Agent Comparisons**
  This study compared LAMBDA with ChatGPT and GitHub Copilot, both of which lack structured orchestration. Future work could extend this analysis to other multi-agent systems that incorporate planning, memory, or dynamic coordination in order to better understand how specific architectural choices influence analytical behavior.

These directions preserve LAMBDA's strength in reliable execution while offering paths to increase its adaptability and contextual fluency.

7. Conclusion

This study systematically evaluated LAMBDA, a multi-agent data analysis system, alongside ChatGPT and GitHub Copilot. Rather than measuring output accuracy alone, we examined how architecture shapes consistency, modeling decisions, and responsiveness to user instructions.

LAMBDA consistently generated complete and reproducible pipelines across runs. Its agentic design, structured around modular roles and validation-driven execution, enabled stable outcomes without user intervention. However, the system did not adapt to intermediate outputs or adjust its behavior based on prompt phrasing. ChatGPT, by comparison, was more responsive to context and follow-up input but exhibited greater variability and required more user guidance. GitHub Copilot performed well for local code generation but lacked orchestration and struggled with multi-step workflows in notebook environments.

Overall, our findings suggest that LAMBDA is best suited for workflows where reproducibility, transparency, and controlled execution are priorities. This includes educational use, auditing contexts, and structured analytical pipelines. For exploratory tasks or fast-paced iteration, tools like ChatGPT may offer more flexibility. GitHub Copilot is better suited for structured, file-based projects where users define the workflow through organized code files and comments, and where execution is managed outside of notebook environments. Taken together, these comparisons underscore that differences in analytical behavior are not merely a function of model capability, but of system design choices around orchestration, execution, and control.

8. Student Assignment: Evaluating LAMBDA's Behavior on a Real-World Analytical Task

Overview

This assignment invites you to explore how LAMBDA behaves as a multi-agent analytical assistant. You will define a real-world data analysis task, guide LAMBDA through the workflow using natural language prompts, and analyze its behavior at each stage.

No manual coding is required; all tasks should be performed via LAMBDA's Gradio-based interface. Your final deliverable should be a 6–9 page report that balances documentation with interpretation.

Part 1: Define Your Task

Choose a dataset and formulate a question that requires multiple steps of analysis. Good tasks include combinations of data loading, cleaning, modeling, and interpretation.

Your task definition should address:

- **What** analytical question you are trying to answer
- **Why** this task benefits from AI assistance
- **What** success would look like (i.e., specific outputs, insights, or conclusions)

Keep the task moderately scoped — it should be complex enough to surface interesting system behavior, but not so large that it becomes unmanageable.

Part 2: Interact with LAMBDA

Use natural language prompts to walk LAMBDA through the task. You may retry or rephrase prompts, but do not manually alter the code it generates. As you interact, document both helpful and problematic system behaviors.

You should include:

- **2–3 excerpts** showing correct, insightful, or proactive behavior
- **2–3 excerpts** where LAMBDA misunderstood, repeated steps, or defaulted inappropriately

In each case, briefly explain what happened and what made the behavior notable.

Part 3: Analyze System Behavior

Go beyond describing outcomes. For each success or failure, provide a hypothesis about **why** LAMBDA behaved the way it did. Your analysis can draw on:

- Agent roles (e.g., Orchestrator, Programmer, Validator)
- Architectural constraints (e.g., no memory across prompts, fixed templates)
- System goals (e.g., prioritize execution success over flexibility)

You do not need to cite code, but your explanations should reflect a mechanistic understanding based on the project report.

Part 4: Recommend Improvements

Based on your interaction, propose **1–2 concrete improvements** that would make LAMBDA more useful or robust. These should respond to specific pain points you encountered.

Examples include:

- Adding a memory summary across prompts
- Offering validation previews before execution
- Making model selection assumptions visible
- Letting users approve next steps more explicitly

Avoid vague suggestions. Each recommendation should be tied to actual evidence from your interaction.

Part 5: Meta-Reflection: What Collaboration Looks Like

Reflect on your overall experience. Use this section to think critically about human–AI collaboration, not just tool performance.

Questions to consider:

- Did LAMBDA feel more like a tool, a partner, or a research assistant?
- When was its initiative helpful, and when was it counterproductive?
- What frustrated or surprised you, and how did you adjust?
- What would LAMBDA need to change for you to trust it more as an autonomous agent?

This section should be thoughtful but concise; 1–2 paragraphs are fine.

Submission Guidelines

- Submit a 6–9 page PDF report
- Include clear section headers for each part
- Embed screenshots or transcripts as evidence (or place them in an appendix)
- You may work individually or in pairs (each team must choose a different dataset)

Component	Focus	Weight
Task Definition	Clarity and relevance of the analytical goal	15%
Interaction Depth	Variety and depth of system use	25%
Behavioral Analysis	Thoughtful interpretation of system behavior	25%
Recommendations	Actionable suggestions grounded in experience	20%
Reflection	Insight into human–AI collaboration	15%

References

Wang, L., et al. (2023). *A Survey on Large Language Model Based Autonomous Agents*. arXiv. https://arxiv.org/abs/2308.11432

Tran, K.-T., et al. (2025). *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*. arXiv. https://arxiv.org/abs/2501.06322

Sun, H., et al. (2024). *LAMBDA: A Foundation Agent for Structured Data Analysis*. arXiv. https://arxiv.org/pdf/2407.17535

Appendix A: Detailed Scoring Justifications

Criterion 1 – Ease of Getting Started

Definition: *How quickly and intuitively the tool begins the analysis without requiring complex setup or lengthy instructions.*

- **LAMBDA** (Score: 3): Getting started with LAMBDA requires cloning a GitHub repository, installing dependencies, and either setting up a Codespaces environment or launching the app locally within a supported code editor. This introduces friction, especially for users unfamiliar with Python environment setup. While the UI becomes intuitive once loaded, setup requires attention to Python versions and `requirements.txt`. Compared to browser-based tools, this represents moderate overhead.
- **ChatGPT** (Score: 5): ChatGPT launches instantly in any browser with no installation, configuration, or file setup. Users can begin prompting immediately. There is no need to manage environments, making it much more accessible, particularly for beginners or those outside a development context.

Criterion 2 – Ease of Iterating Model Choices

Definition: *How easily the underlying model type can be changed without restarting or restructuring the workflow.*

- **LAMBDA** (Score: 2): LAMBDA does not treat model type as a swappable component. Across runs 1–5, changing models (e.g., logistic regression to SVM) requires full re-specification in a task list or prompt. In `lambda_run3`, switching models led to variable name resets (`X_train_mlp`, `train_data`), pipeline redefinition, and preprocessing recomputation. It lacks a one-line way to "just switch models."
- **ChatGPT** (Score: 4): ChatGPT can switch model types mid-sequence by prompting "now try SVM" or "replace with random forest." In `chatgpt_run2`, it accepts the switch and reuses `X_train` and `X_test` without needing a fresh pipeline. Though it may default to old settings unless reminded, model switching is fluid and immediate.

Criterion 3 – Ability to Complete the Task

Definition: *Whether the tool completes the full analytical pipeline correctly and defensibly, from data loading through model training and reporting.*

- **LAMBDA** (Score: 5): LAMBDA consistently completes the entire ML pipeline across runs 1–5. It loads data, applies preprocessing, trains models, evaluates performance, and saves outputs. In `lambda_run4`, LAMBDA even includes grid search and pipeline outputs in its final report. There were no breakpoints or step skips observed across sessions. Model saving and PDF report generation succeed reliably.

- **ChatGPT** (Score: 4): ChatGPT also completes the end-to-end task but exhibits minor slip-ups. For example, in `chatgpt_run3`, it applies a new model to unscaled data after previously applying StandardScaler, indicating that the session did not retain the preprocessing context. While the full sequence can be completed, such redefinition or drift requires monitoring.

Criterion 4 – Statefulness

Definition: *Whether the system keeps the data, models, and other results it created earlier so they can still be used or saved in later steps of the same session.*

- **LAMBDA** (Score: 4): LAMBDA maintains state across a session. In `lambda_run5`, it recalled the trained MLP model and generated a report based on saved results. However, state is lost if the user closes the app or restarts the environment. It also allows saving models explicitly.
- **ChatGPT** (Score: 2): ChatGPT stores conversational context but not actual model or file objects. In `chatgpt_run4`, it could not reuse a trained model from earlier prompts. Any resume attempt requires reinitialization. It remembers logic but lacks persistence.

Criterion 5 – Ease of Tuning Model Parameters

Definition: *How easily hyperparameters or modeling assumptions can be modified without restarting the workflow.*

- **LAMBDA** (Score: 4): LAMBDA handles user-defined GridSearchCV well. In `lambda_run2` and `lambda_run5`, users specified multiple solver options and hidden layer sizes, and LAMBDA executed the search and reported optimal settings clearly. It maintains preprocessing consistency and does not overwrite prior steps. However, edits must be carefully framed for changes to apply.
- **ChatGPT** (Score: 3): ChatGPT accepts tuning instructions, but with volatility. In `chatgpt_run2`, it redefined the model with new parameters but did not always retain prior hyperparameter context, leading to inconsistent layer sizes. It may also rerun the full pipeline unexpectedly. While responsive, tuning with ChatGPT demands user vigilance.

Criterion 6 – Stability (Data Decisions)

Definition: *Consistency of preprocessing decisions (e.g., missing values, encoding, feature inclusion) across repeated runs.*

- **LAMBDA** (Score: 4): LAMBDA makes consistent preprocessing decisions. In `lambda_run1` to `lambda_run4`, it consistently imputes missing values, encodes categorical features using OneHotEncoder, and standardizes numerical features. The structure of the pipeline remains stable unless explicitly changed.
- **ChatGPT** (Score: 3): ChatGPT shows some variability in how it handles preprocessing. In `chatgpt_run4`, it dropped null values, whereas in earlier runs it used imputation. This change

is not flagged or explained, suggesting that phrasing or position in the prompt sequence may lead to different defaults.

Criterion 7 – Stability (Method Choices)

Definition: *Consistency of modeling decisions (e.g., scaling, solver choice, model defaults) across repeated runs under identical prompts.*

- **LAMBDA** (Score: 4): LAMBDA typically uses the same modeling choices unless directed otherwise. For example, it consistently selects `solver='lbfgs'` for logistic regression and applies standard scaling across different runs. It avoids unexpected shifts in solvers, activations, or feature selection.
- **ChatGPT** (Score: 3): ChatGPT is more sensitive to prompt phrasing. In `chatgpt_run3`, it changed solver defaults compared to `chatgpt_run1`, without a prompt-driven reason. These small variations may reflect underlying randomness or default fallback behavior.

Criterion 8 – Continuity Across Steps

Definition: *The ability to build later steps directly on earlier ones within a single workflow, without redoing or contradicting previous work.*

- **LAMBDA** (Score: 5): LAMBDA reliably chains its steps. In `lambda_run2` and `lambda_run4`, feature selection, scaling, model training, and tuning are passed along cleanly to downstream steps. It consistently references prior splits and processed data without recomputation, as long as the session is live.
- **ChatGPT** (Score: 4): ChatGPT generally maintains continuity but can silently forget earlier choices. In `chatgpt_run3`, it reapplied preprocessing steps without checking if they had already been completed. This does not break the pipeline but introduces risk of duplication.

Criterion 9 – Transparency / Auditability

Definition: *The degree to which the system shows what it did, why it did it, and what intermediate states looked like.*

- **LAMBDA** (Score: 3): LAMBDA logs each modeling and preprocessing step with clear labels. However, it does not explain *why* a step was taken unless prompted. Much of the transparency is deferred to its report output rather than embedded in-session. Users must infer rationale from logs.
- **ChatGPT** (Score: 4): ChatGPT often provides inline reasoning. In `chatgpt_run2`, it explains why certain preprocessing steps (e.g., normalization) were applied. These insights are embedded during execution, which helps with live auditing.

Criterion 10 – Interpretability of Outputs

Definition: *Clarity of explanations, code structure, and reporting for a human reader.*

- **LAMBDA** (Score: 4): LAMBDA outputs are clean and labeled. Evaluation metrics are formatted clearly in its final report, which includes model parameters and scores. However, during execution, in-session outputs are more functional than explanatory.
- **ChatGPT** (Score: 4): ChatGPT enhances interpretability by explaining metrics inline. In `chatgpt_run1`, it describes accuracy and confusion matrix values with narrative. However, formatting varies, and summaries are not always grouped systematically.